

Functions, Classes in Python



Agenda

~~☐ What is Python? Why Python? How to use it?~~

~~☐ Python basics~~

~~☐ Lists~~

~~☐ Tuples~~

~~☐ Dictionary and sets~~

☐ **Functions**

☐ **Classes**

☐ **Try-Catch exception**

Agenda

 ~~What is Python? Why Python? How to use it?~~

 ~~Python basics~~

 ~~Lists~~

 ~~Tuples~~

 ~~Dictionary and sets~~

 **Functions**

 **Classes**

 **Try-Catch exception**

About Functions in Python

What is a Function?

- A function is a block of code which only **runs when it is called**.
- It's a fundamental concept in programming that helps you write **cleaner**, more **efficient**, and more readable code.

Why use Function?

- **Reusability:** Once you write a function, you can call it (or "use" it) as many times as you want, from different parts of your code, without having to rewrite the same lines.
- **Readability:** A well-named function (like `calculate_total` or `find_highest_score`) makes the purpose of your code clear to anyone who reads it, including yourself.

How to use them?

```
def greeting(name):  
    print(f"Hello, {name}!")  
✓ 0.0s
```

```
greeting("Alice")  
✓ 0.0s  
Hello, Alice!
```

```
greeting("William")  
✓ 0.0s  
Hello, William!
```

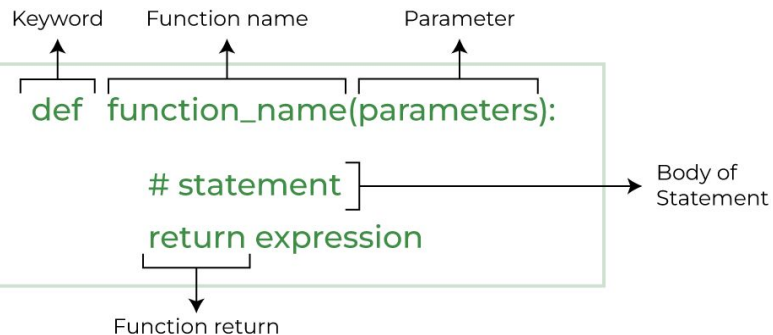
Function declaration

You create use keyword **def** to create a function.

```
def greeting(name):  
    greet_txt = print(f"Hello, {name}!")  
    return greet_txt
```



```
greeting("Alice")  
#Outputs: Hello, Alice!
```



Positional Arguments

Positional Arguments: The simplest way to pass arguments is by their position.

A positional argument is an argument passed to a function based on its position/order, not by name. Python matches each value to a parameter simply by where it appears in the function call.

```
def create_person(name, age):  
    print(f"Name: {name}, Age: {age}")  
create_person("Alice", 30) #Positional Arguments  
# Output: Name: Alice, Age: 30  
create_person(30, "Alice")  
# Output: Name: 30, Age: Alice - This is wrong!
```

Keyword Arguments

Keyword Arguments: With keyword arguments, you explicitly name each argument when you call the function.

```
def create_person(name, age):  
    print(f"Name: {name}, Age: {age}")  
# The order doesn't matter here  
create_person(age=30, name="Alice") #Keyword Arguments  
# Output: Name: Alice, Age: 30
```

```
create_person(age=30, name="Alice", email ="alice@gmail.com")
```

`TypeError: create_person() got an unexpected keyword argument 'email'`

Arbitrary Arguments (*args, **kwargs)

- **Arbitrary arguments**, allow a function to accept an **unspecified number of arguments**.
- This is useful when want to create a function that can accept unknown number of arguments.
- Python handles this with two special syntaxes: ***args** and ****kwargs**.

Arbitrary Positional Arguments: *args

***args** allows the function to accept any number of positional arguments.

```
def sum_all(*numbers):  
    # 'numbers' is a tuple  
    print(f"I received a tuple:  
{numbers}")  
    total = 0  
    for num in numbers:  
        total += num  
    return total
```

```
print(sum_all(1, 2, 3))  
# Output:  
# I received a tuple: (1, 2, 3)  
# 6  
print(sum_all(5, 10))  
# Output:  
# I received a tuple: (5, 10)  
# 15
```

Arbitrary Arguments (*args, **kwargs)

Arbitrary Keyword Arguments: **kwargs

****kwargs** allows the function to accept any number of keyword arguments.

****kwargs** is used to accept extra arguments that are passed with a **key** (a name) and a **value**.

```
def create_profile(**details):  
    # 'details' is a dictionary containing all the keyword arguments  
    print(f"I received a dictionary: {details}")  
    for key, value in details.items():  
        print(f"{key.capitalize()}: {value}")
```

```
create_profile(name="Charlie", age=45)  
# Output:  
# I received a dictionary: {'name': 'Charlie', 'age': 45}  
# Name: Charlie  
# Age: 45
```

```
create_profile(name="Alice", age=30, city="New York")
```

```
create_profile(name="Bob", age=25, city="London", country="UK",  
email="bob@example.com")
```

Pass 2 Key Value pairs

Pass 3

Pass 5

Default Parameter in function

Default parameters allow you to set a **predefined value** for a function's argument. This makes the **argument optional** when you call the function. If you don't provide a value for that argument, Python will automatically use the default value. If you do provide a value, it will **override the default**.

```
def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")
```



Default parameter

```
# Usage with a default value  
greet("Alice")  
# Output: Hello, Alice!
```



Not passing Greeting

```
# Usage by overriding the default value  
greet("Bob", greeting="Good morning")  
# Output: Good morning, Bob!
```



Pass custom
Greeting

QUIZ Time

Time Duration: 6 Mins

Instructions: Complete the following three tasks by writing a Python function for each.

1. **Palindrome Checker (A palindrome is a phrase that reads the same forwards and backward ("racecar"))**
 - Create a function called `is_palindrome` that takes a single string as input.
 - The function should return `True` if the string is a palindrome and `False` otherwise.
 - Check with "hello" and "racecar"

2. **Remove Duplicates**
 - `my_numbers = [1, 2, 2, 3, 4, 4, 5]`
 - Create a function called `remove_duplicates` that accepts a list of numbers "my_numbers".
 - The function should remove any duplicate numbers and return the list with only unique elements.

3. **Even Number Filter (BONUS*)**
 - Create a function called `get_even_numbers` that can accept **any number of integer arguments**.
 - The function should return a new list containing only the even numbers from the arguments provided.
 - *You may need to use a special syntax to handle the multiple arguments.*

Hint: Use `*args` to accept any of integer arguments.

QUIZ Time

12

SOLUTION

#1. Palindrome Checker

```
def is_palindrome(input_string):  
    reversed_string = input_string[::-1] # Reverse the input string  
    return input_string == reversed_string
```

```
print(is_palindrome("racecar"))  
print(is_palindrome("hello"))
```

#OUTPUT:

```
# True  
# False
```

#2 Remove Duplicates

```
def remove_duplicates(input_list):  
    unique_set = set(input_list) #making set removes duplicates  
    unique_list = list(unique_set) #converting back to list  
    return unique_list
```

```
my_numbers = [1, 2, 2, 3, 4, 4, 5]  
unique_numbers = remove_duplicates(my_numbers)  
print(unique_numbers)
```

#OUTPUT:

```
# [1, 2, 3, 4, 5]
```

QUIZ Time

13

SOLUTION

#3. Even Number Filter (BONUS*)

```
def get_even_numbers(*args):  
    even_numbers = [] # Create an empty list to store even numbers  
    for number in args:  
        if number % 2 == 0: # Check if the number is even  
            even_numbers.append(number) # if yes, append to the list  
    return even_numbers  
print(get_even_numbers(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
```

#OUTPUT:

```
# [2, 4, 6, 8, 10]
```

Agenda

 ~~What is Python? Why Python? How to use it?~~

 ~~Python basics~~

 ~~Lists~~

 ~~Tuples~~

 ~~Dictionary and sets~~

 **Functions**

 **Classes**

 **Try-Catch exception**

About Classes in Python

What is a Class?

- Classes in Python are a basic part of **Object-Oriented Programming (OOP)**.
- A class is a blueprint for creating objects.
- They act as templates for creating objects, which can have their own **variables** (called **attributes**) and **functions** (called **methods**).

Lets understand class using an example!!

About Classes in Python

Imagine you're describing a **student**. Every student has:

- a name
- an age

And every student can **do** something:

- introduce themselves

A class is just a way to group "things a student **has**" and "things a student **can do**" into one bundle.

A class is NOT a real student. It's just a **template/blueprint**

```
class Student:  
    pass
```

Right now, this class does nothing. It's an empty blueprint. No student exists yet.

About Classes in Python

Let's say every student needs a `name` and an `age`. We tell the class this using a special method called `__init__`:

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

`__init__` runs **automatically** the moment you create a student. It fills in the blank fields (`name`, `age`) for that one student.

What is `self`?

`self` just means **"this specific student"**, as opposed to some other student.

So `self.name = name` means: *"Take the `name` value that was passed in, and store it onto this particular student."*

Now let's use the blueprint to make a **real** student:

```
student1 = Student("Alice", 21)
```

1. Python creates a new, empty student object.
2. Python automatically calls `__init__`, passing in `self` (this new empty student), `"Alice"`, and `21`.
3. Inside `__init__`, `self.name = name` runs → this new student's name gets set to `"Alice"`.
4. Same for age → gets set to `21`.
5. The finished student (now with name + age filled in) gets stored in the variable `student1`.

Class Initialization

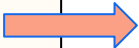
By convention, class names should **start with a capital letter**.

(e.g., Mobile, Car, Person)

- The **`__init__()` method** is where you assign values to the object's attributes.
- In the example below, `self.brand` and `self.model` become attributes of the Mobile object.

#Create a new class called Mobile

class Mobile:



```
def __init__(self, brand, model):
```

```
    self.brand = brand # Attribute Brand
```

```
    self.model = model # Attribute Model
```



```
some_phone = Mobile("Apple", "iPhone 16")
```

```
print(some_phone.brand) # Output: Apple
```

```
print(some_phone.model) # Output: iPhone 16
```

Class Methods

- Class methods are **functions that belong to the class**.
- The first parameter of a class method is always self, which provides access to the object's own attributes.
- Other parameters can be passed into the method just like with a regular function.

```
class Mobile:
    def __init__(self, brand, model): #Initialization
        self.brand = brand
        self.model = model

    def describe(self, price): # Method to describe the mobile
        print(f"This {self.brand} {self.model} is priced at
{price} USD.")

some_phone = Mobile("Apple", "iPhone 16") # Create an instance
some_phone.describe(999) # Call the describe method
# Output: This Apple iPhone 16 is priced at 999 USD.
```

Take Home Quiz

Do the following:

1. Define a class called Circle, when initializing this class object, it accepts two values as class attributes called radius and color.
2. Add two methods to calculate the area and circumference of the circle.
3. Define a Rectangle class. It has methods to calculate the area and perimeter.

Take Home Quiz

22

SOLUTION: 1) Define a class called Circle, when initializing this class object, it accepts two values as class attributes called radius and color.

```
class Circle:
    def __init__(self, radius=1, color='red'):
        self.radius = radius
        self.color = color
```

```
class Circle:
    def __init__(self, radius=1, color='red'):
        self.radius = radius
        self.color = color
```

```
c = Circle(radius=5, color="black")
print(f"This circle is {c.color}")
```

✓ 0.0s

This circle is black

Take Home Quiz

23

SOLUTION: 2) Add two methods to calculate the area and circumference of the circle.

```
class Circle:
    def __init__(self, radius=1, color='red'):
        self.radius = radius
        self.color = color
        self.PI = 3.14
    def get_circumference(self):
        return 2 * self.PI * self.radius
    def get_area(self):
        return self.PI * (self.radius ** 2)
```

```
class Circle:
    def __init__(self, radius=1, color='red'):
        self.radius = radius
        self.color = color
        self.PI = 3.14
    def get_circumference(self):
        return 2 * self.PI * self.radius
    def get_area(self):
        return self.PI * (self.radius ** 2)
```

```
c = Circle(radius=5, color="black")
print(c.get_circumference())
print(c.get_area())
```

✓ 0.0s

31.400000000000002

78.5

Take Home Quiz

24

SOLUTION: 3) Define a Rectangle class. It has methods to calculate the area and perimeter.

```
class Rectangle:
    def __init__(self, w=1, h=1):
        self.w = w
        self.h = h
    def getArea(self):
        return self.w * self.h
    def getPerimeter(self):
        return 2 * self.w + 2 * self.h
```

```
class Rectangle:
    def __init__(self, w=1, h=1):
        self.w = w
        self.h = h
    def getArea(self):
        return self.w * self.h
    def getPerimeter(self):
        return 2 * self.w + 2 * self.h

r = Rectangle(w=2, h=3)
print(r.getPerimeter())
print(r.getArea())
```

✓ 0.0s

10

6

Agenda

 ~~What is Python? Why Python? How to use it?~~

 ~~Python basics~~

 ~~Lists~~

 ~~Tuples~~

 ~~Dictionary and sets~~

 **Functions**

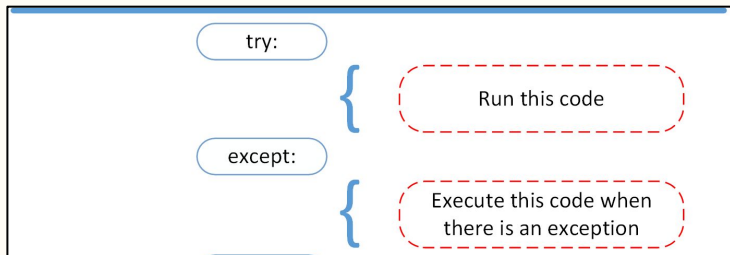
 **Classes**

 **Try-Catch exception**

About Try-Catch/ Try-Except in Python

What is a Try-Catch/ Try-Except?

- It is a form of Exception Handling



Try-Except Example

```
# A program without exception handling
# This will crash if the user enters 0
num = int(input("Enter a number: "))
result = 10 / num
print(result)
```

```
# Output (if user enters 0):
# ZeroDivisionError: division by zero
```

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)
except ZeroDivisionError:
    print("Error: You cannot divide by zero.")
```

```
# Output (if user enters 0):
# Error: You cannot divide by zero.
# The program continues to run without crashing.
```

More Reading on Try-Except

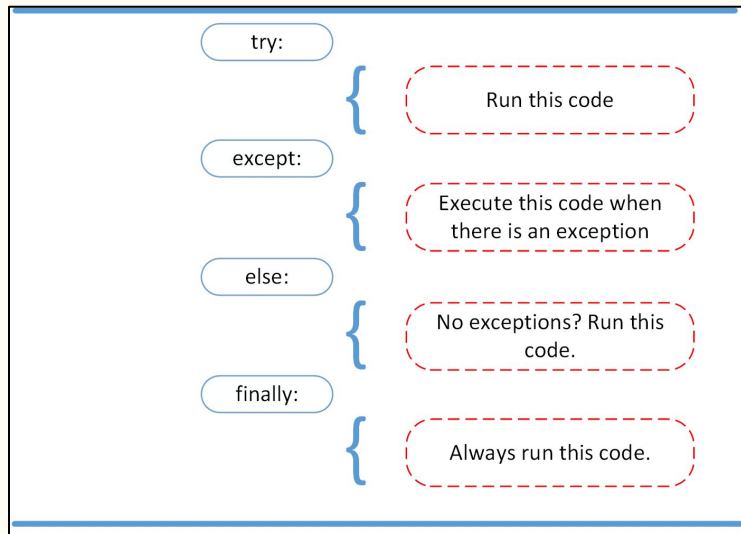
- Error handling: https://www.w3schools.com/python/python_try_except.asp
- File handling: https://www.w3schools.com/python/python_file_handling.asp

The **try** block lets you test a block of code for errors.

The **except** block lets you handle the error.

The **else** block lets you execute code when there is no error.

The **finally** block lets you execute code, regardless of the result of the try- and except blocks.



Homework Name: Lab1HW4

29

1. Library Management System (5 Points)

Instructions:

1. Create a simple library management system using two classes: **Book** and **Library**.
2. Show how to create a Book object, a Library object, add a book, borrow a book, and display the available books.

Hint:

Class **Book**

- **Attributes:**

- **title** (string)
- **author** (string)
- **is_available** (boolean; **True** by default)

- **Methods:**

- The **__init__** method should initialize the **title**, **author**, and set **is_available** to **True**.

Class **Library**

- **Attributes:**

- **books** (a list that will store **Book** objects)

- **Methods:**

- **__init__**: Initializes the **books** list as empty.
- **add_book(book)**: Adds a new **Book** object to the **books** list.
- **borrow_book(title)**: Finds a book by its **title** and changes its **is_available** status to **False**. It should handle cases where the book is not found or is already borrowed.
- **return_book(title)**: Finds a book by its **title** and changes its **is_available** status to **True**. It should handle cases where the book is not found.
- **display_available_books()**: Prints the **title** and **author** of all books that are currently available.

Homework Instructions

Submission Format:

- Please submit your assignments as either a **.ipynb** (Jupyter Notebook) file or a **.pdf** file.
- If you choose to submit a **.pdf**, make sure all of your code and its output are **clearly visible**.

File Naming Convention:

- You must name your file exactly as shown below:
- YourName_YourStudentID_AssignmentName.FileExtension
- **Example:** Anushka_st126222_Lab4HW.pdf

Late Submissions:

- Please submit all assignments **on time**. Late submissions will be **penalized**.
- Submissions made **within 24 hours after the deadline** will receive **50% penalty**.
- Submissions will **not be accepted** more than **24 hours after the deadline**.

Resource for Further Studies into Python



<https://www.w3schools.com/python/>

Any Questions?

Send Email at: st126222@ait.ac.th